



CART NINJA

Shopify Cart Optimization App

A Complete Technical & Functional Project Report

Prepared By	Abilashmi.K
Company / Organization	Digifyce
Department	Tech
Application Type	Shopify Public App (Theme App Extension)
Technology Stack	React · Node.js · Prisma · Shopify API v2025-10
Document Version	2.0
Date	April 2026

ABSTRACT / EXECUTIVE SUMMARY

Cart Ninja is a feature-rich Shopify application engineered to dramatically improve the cart and checkout experience for online stores. By embedding a fully customizable Cart Drawer directly into the storefront via a Shopify Theme App Extension, Cart Ninja enables merchants to display dynamic upsell recommendations, apply coupon codes, visualize milestone-based progress bars, and track shopper behavior — all within a single, cohesive interface.

The application is built on a modern full-stack architecture that combines **React Router v7** (server-side rendering with React), the **Shopify Polaris** design system for the merchant admin UI, and **Prisma ORM** backed by SQLite for development-time persistence. The storefront extension is implemented as a lightweight Vanilla JavaScript widget injected via Shopify's Theme App Extension system, ensuring zero performance overhead on store load times.

Key capabilities of Cart Ninja include: (1) a real-time Cart Drawer with configurable progress-bar milestones and reward tiers; (2) a Coupon Engine that surfaces Shopify native discount codes within the cart; (3) a rule-based and AI-assisted Upsell Engine; (4) a Frequently Bought Together (FBT) combo widget; (5) an Analytics Dashboard that tracks checkout clicks, coupon applications, upsell revenue, and total cart revenue; and (6) a flexible Subscription Plan management page.

All merchant-facing configurations are persisted in a dual-storage system: a local JSON file and a remote PHP backend, providing resilience against external API downtime. Shopify webhooks (orders/create, app/uninstalled, app/scopes_update) are handled to keep metrics synchronized in real time.

Key Metrics at a Glance

Attribute	Detail
App Type	Shopify Public App with Theme App Extension
Shopify API Version	October 2025 (2025-10)
Primary Language	JavaScript (ESM) / JSX
Total Route Files	25+ (API + UI routes)
Extension Assets	Cart Drawer, Coupon Slider, FBT Widget

Database	SQLite (dev) / Any Prisma-compatible DB (prod)
----------	--

TABLE OF CONTENTS

1. Introduction	6
1.1 Purpose of the Application	6
1.2 Problem Statement	6
1.3 Objectives	6
2. System Overview	8
2.1 How Cart Ninja Works	8
2.2 Key Features Summary	8
3. Technology Stack	10
3.1 Frontend	10
3.2 Backend	10
3.3 Shopify APIs	10
3.4 Tools & Libraries	11
4. System Architecture	12
4.1 Architecture Overview	12
4.2 Layer Descriptions	12
4.3 Architecture Diagram	12
5. Database Design	14
5.1 Models & Tables	14
5.2 Field Descriptions	14
5.3 Relationships	15
6. File Structure	17
7. Modules / Features Description	19
7.1 Cart Drawer	19
7.2 Coupon Engine	19
7.3 Upsell Engine	19
7.4 FBT / Combo Builder	20
7.5 Analytics Dashboard	20
7.6 Admin Dashboard	20
7.7 Subscription Plans	20
8. UI Screens	21
9. Workflow / User Flow	25
10. API Design	27
11. Shopify App Review & Verification Process	30
12. OAuth Authentication Architecture	30
13. HMAC Validation & Webhook Security	32
14. Manual QA Testing Process	32
15. AI-Assisted Testing Workflow	34
16. Deployment Validation Process	34
17. Security Testing Procedures	36
18. Future Developer Onboarding Guide	36
19. Final Shopify Submission Checklist	36
20. Security Considerations	38

21. Performance Optimization	40
22. Challenges Faced	42
23. Future Enhancements	43
24. Conclusion	44
25. References	44

1. INTRODUCTION

1.1 Purpose of the Application

Cart Ninja is a Shopify application designed to help e-commerce merchants maximize their revenue from every customer session. The app enhances the standard Shopify cart experience by replacing or augmenting the default cart UI with an intelligent, customizable Cart Drawer — a slide-in panel that appears when a shopper adds an item to their cart.

The purpose extends beyond visual enhancement. Cart Ninja serves as a conversion optimization platform, enabling merchants to apply upsell strategies, surface discount codes, motivate shoppers with reward milestones, and gain actionable analytics — all from a single, unified admin interface built natively inside Shopify's admin panel.

1.2 Problem Statement

The cart page is one of the most critical but underutilized moments in the e-commerce funnel. Many Shopify stores lose potential revenue at the cart stage due to:

- No mechanism to show relevant product recommendations inside the cart.
- Coupon codes buried deep in the checkout flow, causing friction.
- No visual incentive (e.g., "Add \$20 more to get free shipping") to increase average order value.
- Lack of analytics specific to the cart interaction — merchants cannot track what shoppers do in the cart.
- Disconnected tools requiring multiple apps to achieve what Cart Ninja provides in one.

Cart Ninja solves all of the above with a unified, embedded solution that operates entirely within Shopify's native infrastructure.

1.3 Objectives

- Build a fully configurable Cart Drawer injected via Shopify Theme App Extension.
 - Develop a Coupon Engine that pulls live Shopify discount codes and presents them in-cart.
 - Create a rule-based and AI-assisted Upsell Engine with multiple display layouts.
 - Implement a milestone-based Progress Bar to motivate shoppers toward rewards.
 - Provide a Frequently Bought Together (FBT) / Combo product widget.
 - Build a real-time Analytics Dashboard tracking key cart engagement metrics.
 - Support subscription-based billing tiers (Free, Pro, Enterprise).
 - Ensure zero-friction Shopify OAuth installation and session management.
-

2. SYSTEM OVERVIEW

2.1 How Cart Ninja Works

Cart Ninja operates as two interconnected parts: the **Merchant Admin App** (running inside Shopify Admin) and the **Storefront Extension** (running on the customer-facing store). The merchant configures all settings through the admin app; those settings are saved to the backend and served to the storefront extension in real time via a configuration API.

When a shopper on the live store adds a product to their cart, the Cart Drawer extension intercepts the cart update event, fetches the latest configuration from Cart Ninja's API, and renders the drawer with the appropriate upsell products, coupon codes, and progress bar — all without a page reload. Every meaningful interaction (coupon click, upsell click, checkout click) is tracked and sent to the analytics backend.

2.2 Key Features Summary

Cart Drawer (Storefront Extension)

A fully customizable slide-in cart panel powered by a Shopify Theme App Extension. Supports progress bar milestones, coupon display, upsell products, and FBT combos. Configurable colors, layout, placement, and text from the admin UI.

Coupon Engine

Fetches active Shopify discount codes via the Admin GraphQL API and displays them as styled cards inside the Cart Drawer. Supports multiple visual styles, position configuration (top/bottom), and layout modes (grid/list). Allows merchants to create new discount codes directly from Cart Ninja.

Upsell Engine (Rule-Based + AI)

Supports four rule types: GLOBAL (show all shoppers), TRIGGERED (based on cart items), GLOBAL_EXCEPT (exclusions), and CART_CONDITIONS (cart value threshold). AI mode uses product context to suggest relevant upsells. Multiple display layouts: slider and vertical grid.

Progress Bar / Milestone System

Configurable multi-tier progress bar that motivates shoppers with rewards (free shipping, gifts, discounts) at defined cart value or quantity thresholds. Supports custom icons (preset SVG icons or custom SVG upload), colors, gradients, and placement.

Analytics Dashboard

Tracks: checkout clicks, coupon clicks, upsell clicks, upsell-generated revenue, total cart revenue, and coupon applications. Displays data via interactive area charts, bar charts, pie charts, and composed charts powered by Recharts. Supports date range filtering.

Subscription Plans

Three-tier pricing model (Free, Pro, Enterprise) managed via Shopify Billing API. Plan card UI with feature comparison matrix and upgrade/downgrade flow.

3. TECHNOLOGY STACK

3.1 Frontend (Admin UI)

Technology	Version	Purpose
React	18.3.x	Component-based UI rendering for admin dashboard
React Router (v7)	7.12.x	Full-stack framework with SSR; replaces Remix
Shopify Polaris	13.9.x	Official Shopify design system (components, icons, tokens)
@shopify/app-bridge-react	4.2.x	Shopify App Bridge for embedded app communication

3.2 Backend (Server Layer)

Technology	Version	Purpose
Node.js	≥20.19 / ≥22.12	JavaScript runtime for the server
React Router (v7) Server	7.12.x	Loader/action pattern for SSR data fetching
Prisma ORM	6.16.x	Type-safe database access, migrations, schema
SQLite	dev.sqlite	Development database (upgradable to PostgreSQL/MySQL)
PHP Backend (external)	—	Remote persistence for cart drawer config & analytics
@shopify/shopify-app-react-router	1.1.x	Shopify OAuth, session, webhook handling
PrismaSessionStorage	8.0.x	Stores Shopify OAuth sessions in Prisma DB

3.3 Shopify APIs

API	Usage
Admin GraphQL API (2025-10)	Fetch products, discounts, collections; create/delete discount codes; billing
Storefront API	Cart operations from the client-side extension
Webhooks API	orders/create, app/uninstalled, app/scopes_update events
Shopify Theme App Extension	Inject Cart Drawer block into store themes
Shopify Billing API	Subscription plan management

3.4 Developer Tools & Libraries

Tool	Purpose
Vite 6	Frontend build tool and dev server bundler
plugins	Import rules
GraphQL Codegen	Auto-generate TypeScript types from GraphQL queries
Shopify CLI	App scaffolding, dev tunnel, deploy, config linking
Docker	Containerized deployment support

4. SYSTEM ARCHITECTURE

4.1 Architecture Overview

Cart Ninja follows a **three-tier layered architecture** with an additional Shopify Integration Layer. The system is split between a *Merchant-facing Admin App* (embedded inside Shopify Admin) and a *Customer-facing Storefront Extension*. Data flows bidirectionally between these two sides via Cart Ninja's REST-style API routes.

4.2 Layer Descriptions

Layer 1 — Frontend (Presentation) Layer

All admin UI pages are React components rendered via React Router v7's SSR pipeline. The Shopify Polaris design system provides the component library. Pages are organized as file-based routes under `app/routes/`. The storefront extension is a set of lightweight Vanilla JS + CSS files served directly from the Shopify CDN.

Layer 2 — Backend (Application) Layer

Route loader and action functions act as the API layer. Loaders handle GET requests (data fetching), while actions handle POST/DELETE mutations. Authentication is enforced on every server-side route via `authenticate.admin(request)` from the Shopify app library. Webhooks are handled by dedicated route files.

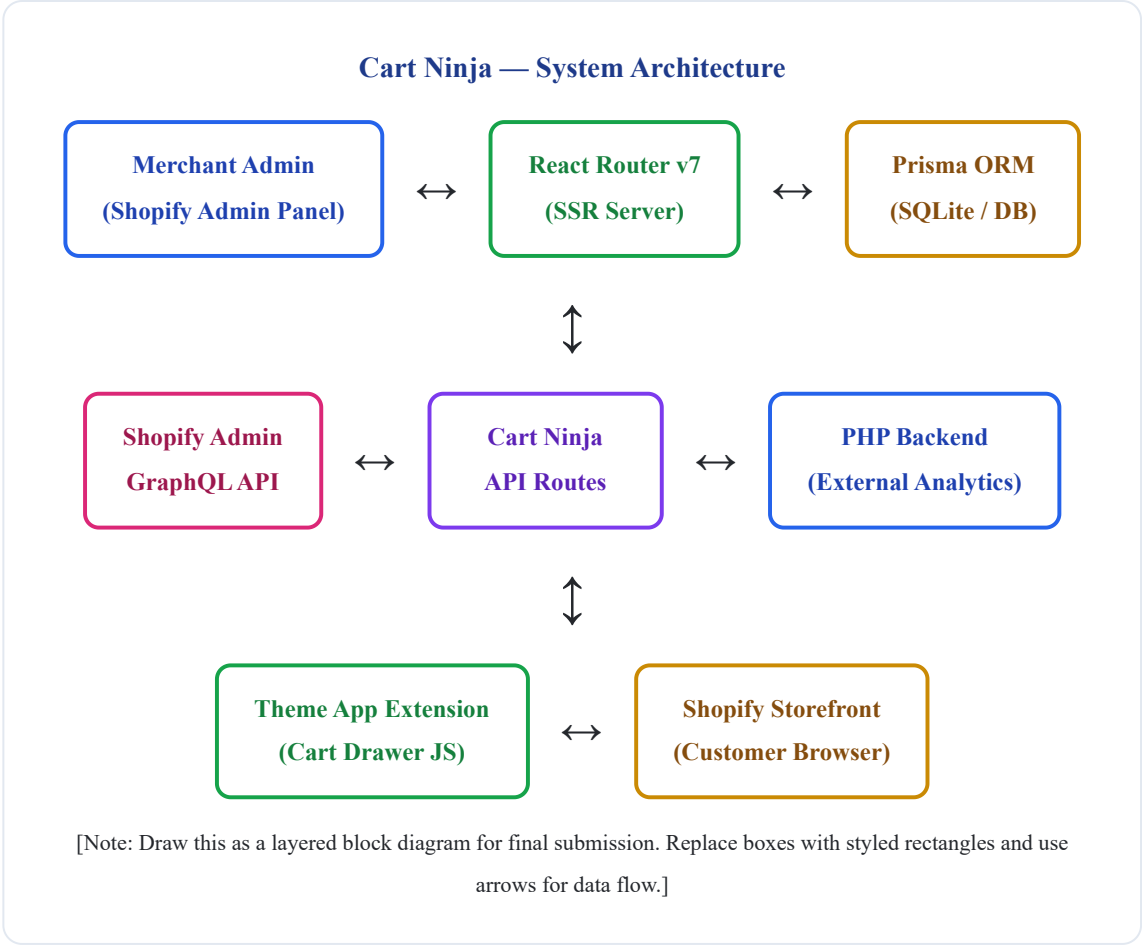
Layer 3 — Database (Persistence) Layer

Prisma ORM manages the database schema and migrations. Three models are defined: `Session` (OAuth sessions), `UpsellRule` (per-shop upsell configuration), and `WidgetSettings` (cart drawer widget JSON configuration). An external PHP backend provides secondary persistence for cart drawer settings and analytics metrics.

Layer 4 — Shopify Integration Layer

Shopify OAuth is handled by `shopify.server.js`. The Admin GraphQL API (v2025-10) is called for product fetching, discount management, and billing. The Theme App Extension is deployed separately via Shopify CLI and communicates with Cart Ninja's API endpoints at runtime.

4.3 Architecture Diagram



5. DATABASE DESIGN

5.1 Models & Tables

Cart Ninja uses Prisma ORM with SQLite (development) and supports any Prisma-compatible relational database for production. There are three primary models defined in `prisma/schema.prisma`:

5.1.1 Session Table

Field	Type	Description
id	String (PK)	Unique session identifier
shop	String	Shopify store domain (e.g., mystore.myshopify.com)
state	String	OAuth state token for CSRF protection
isOnline	Boolean	Whether this is an online (user) or offline (app) session
accessToken	String	Shopify OAuth access token for API calls
scope	String?	Granted permission scopes
expires	DateTime?	Token expiration timestamp
userId	BigInt?	Shopify user ID (for online sessions)
email / firstName / lastName	String?	Staff account details
refreshToken	String?	OAuth refresh token (expiring access token flow)

5.1.2 UpsellRule Table

Field	Type	Description
id	String (PK)	UUID auto-generated
shop	String (indexed)	Shop domain — scopes rules per merchant

name	String?	Human-readable rule name
enabled	Boolean	Activate/deactivate rule without deletion
ruleType	String (indexed)	GLOBAL TRIGGERED GLOBAL_EXCEPT CART_CONDITIONS
priority	Int	Rule evaluation order (higher = evaluated first)
triggerProducts	String? (JSON)	Array of product IDs that trigger this rule
triggerCollections	String? (JSON)	Array of collection IDs as triggers
upsellProducts	String? (JSON)	Array of product IDs to display as upsells
cartValueThreshold	Int?	Minimum cart value to activate CART_CONDITIONS rule
displayLimit	Int	Max number of upsell products to show (default: 3)
layout	String	slider vertical — display format
buttonText	String	CTA button label (default: "Add to Cart")
title	String	Section heading shown in cart drawer

5.1.3 WidgetSettings Table

Field	Type	Description
id	String (PK)	UUID auto-generated
shop	String (unique)	One settings record per shop
coupons	String (JSON)	Coupon display configuration object
fbt	String (JSON)	Frequently Bought Together widget config
progressBar	String? (JSON)	Progress bar tiers, colors, and milestones
upsell	String? (JSON)	Upsell display mode and manual rules config
updatedAt	DateTime	Last configuration save timestamp

5.2 Relationships

All three tables are keyed by `shop` domain string but are intentionally independent (no foreign key constraints) to allow flexible partial configurations. The `UpsellRule` table supports multiple rules per shop (one-to-many relationship between shop and rules), while `WidgetSettings` enforces a unique constraint ensuring one settings record per shop.

6. FILE STRUCTURE

The project follows the React Router v7 (formerly Remix) file-based routing convention with a Shopify extension workspace. Below is the annotated folder tree:

```

cart-app/                                     ← Root of the monorepo workspace
├── app/                                       ← Main application source
│   ├── routes/                              ← All UI + API route handlers
│   │   ├── app._index.jsx                   ← Main Analytics Dashboard (home page)
│   │   ├── app.jsx                          ← Root layout with nav menu & App Bri
│   │   ├── app.cartdrawer.jsx               ← Cart Drawer configuration page
│   │   ├── app.coupons.jsx                  ← Coupon management list page
│   │   ├── app.discounts.create.jsx         ← Discount code creator form
│   │   ├── app.analytics.jsx                ← Detailed analytics page
│   │   ├── app.upsell.jsx                   ← Upsell rule builder page
│   │   ├── app.productwidget.jsx            ← Product widget settings
│   │   ├── app.Plan.jsx                     ← Subscription plan selection page
│   │   ├── app.modern-cartdrawer.jsx        ← Next-gen cart drawer UI (in progres
│   │   ├── app.discount.jsx                 ← Discount overview page
│   │   ├── app.additional.jsx               ← Additional/misc settings page
│   │   ├── api.cartdrawer-config.jsx        ← API: Cart drawer config (GET/POST)
│   │   ├── api.analytics.jsx                ← API: Analytics data proxy
│   │   ├── api.upsell.jsx                   ← API: Upsell rule CRUD
│   │   ├── api.coupons-active.jsx           ← API: Active coupons fetch
│   │   ├── api.coupon-slider.jsx            ← API: Coupon slider widget data
│   │   ├── api.cart-settings.jsx            ← API: Widget settings (GET/POST)
│   │   ├── api.fbt-ai.jsx                   ← API: FBT AI recommendations
│   │   ├── api.ai-upsell.jsx                ← API: AI-powered upsell
│   │   ├── api.product-sample.jsx           ← API: Product sample data
│   │   ├── api.shophandler.jsx              ← API: Shop-level operations
│   │   ├── api.shopify-coupons.jsx          ← API: Shopify native coupons
│   │   ├── api.create_coupon-sample.jsx     ← API: Create sample coupon
│   │   ├── auth.$.jsx                       ← Shopify OAuth callback handler
│   │   ├── auth.login/                      ← Login page
│   │   ├── store.jsx                        ← Store data page
│   │   ├── webhooks.app.uninstalled.jsx     ← Webhook: App uninstalled
│   │   ├── webhooks.app.scopes_update.jsx   ← Webhook: Scope changes
│   │   ├── webhooks.orders.create.jsx       ← Webhook: New order tracking
│   │   └── billing.constants.js             ← Billing plan constants
│   └── components/                          ← Reusable React components
│       ├── AiUpsellSection.jsx              ← AI upsell product selection UI
│       ├── CouponSliderEditor.jsx           ← Coupon slider config editor
│       ├── CurrencyContext.jsx              ← Currency/locale React context
│       └── MilestoneProgressBarPreview.jsx  ← Live preview of progress bar

```

	ProgressBarEditor.jsx	← Progress bar settings editor
	UpsellComponents.jsx	← Shared upsell display components
	UpsellProductEditor.jsx	← Product picker for upsell rules
	api.cart-settings.jsx	← Client-side cart settings API util
	api.cart-settings.shared.js	← Shared config constants & helper
	services/	← Business logic / service layer
	api.upsell.js	← Upsell service helpers
	cartSettings.server.js	← Server-side cart settings logic
	coupons.server.js	← Coupon server logic
	product-widget.server.js	← Product widget server logic
	product-widget.shared.js	← Shared product widget utilities
	storefront-upsell-integration.js	← Storefront upsell bridge
	utils/	← Utility functions
	currency.shared.js	← Currency formatting utilities
	validators/	← Input validation helpers
	styles/	← Global CSS styles
	root.jsx	← Root HTML shell
	entry.server.jsx	← SSR entry point
	shopify.server.js	← Shopify app & OAuth config
	db.server.js	← Prisma client singleton
	extensions/	
	cart-drawer/	← Shopify Theme App Extension
	assets/	
	cart_drawer.js	← Main cart drawer logic
	cart_drawer.css	← Cart drawer styles
	cart_drawer_inline.js	← Inline cart drawer variant
	coupon_slider.js	← Coupon slider widget
	coupon_slider.css	← Coupon slider styles
	fbt_widget.js	← FBT widget script
	fbt_widget.css	← FBT widget styles
	blocks/	← Extension block definitions
	snippets/	← Liquid snippets
	locales/	← i18n translation files
	shopify.extension.toml	← Extension manifest (type: theme)
	prisma/	
	schema.prisma	← Database schema & models
	migrations/	← Auto-generated migration files
	cartdrawer-config-data.json	← Local fallback config storage
	package.json	← Dependencies & npm scripts
	vite.config.js	← Vite build config
	.env	← Environment variables (not committed)

7. MODULES / FEATURES DESCRIPTION

7.1 Cart Drawer Module

Files: `app.cartdrawer.jsx`, `api.cartdrawer-config.jsx`, `extensions/cart-drawer/assets/cart_drawer.js`

The Cart Drawer is the centerpiece of Cart Ninja. It is implemented as a Shopify Theme App Extension block (`type = "theme"`) that merchants activate from their theme editor. The admin configuration page (`app.cartdrawer.jsx`) provides a tabbed interface to configure: the Progress Bar (colors, milestones, reward tiers), Coupon display (styles, position, layout), and Upsell section (mode, rules, layout).

The backend API (`api.cartdrawer-config.jsx`) handles GET/POST requests. On GET, it merges config from the Prisma DB (`WidgetSettings`), the external PHP backend, and defaults. On POST, it saves to both stores and returns the normalized config. A local JSON fallback (`cartdrawer-config-data.json`) ensures resilience if the external API is down.

7.2 Coupon Engine

Files: `app.coupons.jsx`, `app.discounts.create.jsx`, `api.coupons-active.jsx`, `api.shopify-coupons.jsx`

The Coupon Engine fetches Shopify discount codes via the Admin GraphQL API and renders them as styled cards within the Cart Drawer. The `app.coupons.jsx` page lists all coupons with sort/filter controls using Polaris `IndexTable`. Merchants can delete coupons (with GraphQL mutations for `DiscountCodeBasic`, `DiscountCodeBxgy`, and `DiscountCodeFreeShipping` types). The `app.discounts.create.jsx` form supports creating any of the three discount types with full field configuration (value, dates, country restrictions, product/collection applicability).

7.3 Upsell Engine

Files: `api.upsell.jsx`, `api.ai-upsell.jsx`, `app/components/AiUpsellSection.jsx`, `app/components/UpsellProductEditor.jsx`

The Upsell Engine supports both manual and AI-assisted modes. In manual mode, merchants define rules of four types stored in the `UpsellRule` Prisma model. In AI mode, the `api.ai-upsell.jsx` endpoint analyzes cart contents and returns AI-generated product suggestions. The

engine evaluates rules in priority order, applying display limits and layout preferences configured per rule.

7.4 FBT / Combo Builder

Files: `api.fbt-ai.jsx`, `extensions/cart-drawer/assets/fbt_widget.js`

The Frequently Bought Together widget suggests product bundles based on AI or manual configuration. The widget is rendered as a separate asset in the cart drawer extension, showing combo products with a combined "Add All" button to maximize bundle sales.

7.5 Analytics Dashboard

Files: `app._index.jsx`, `app.analytics.jsx`, `api.analytics.jsx`

Tracks six key metrics: checkout clicks, coupon clicks, upsell clicks, upsell revenue generated, total cart revenue, and coupon applications. Data is fetched from an external PHP analytics backend with environment-variable- configured URLs. Visualized with Recharts: AreaChart for revenue trends, BarChart for click comparison, PieChart for metric distribution, and ComposedChart for combined views. Supports date range picker filtering.

7.6 Admin Dashboard (`app._index.jsx`)

The home screen of the merchant admin shows a live summary of all key metrics, quick-action buttons to each module, and banner notifications for setup guidance. Uses Grid, Card, Badge, and ProgressBar from Polaris for a clean dashboard layout.

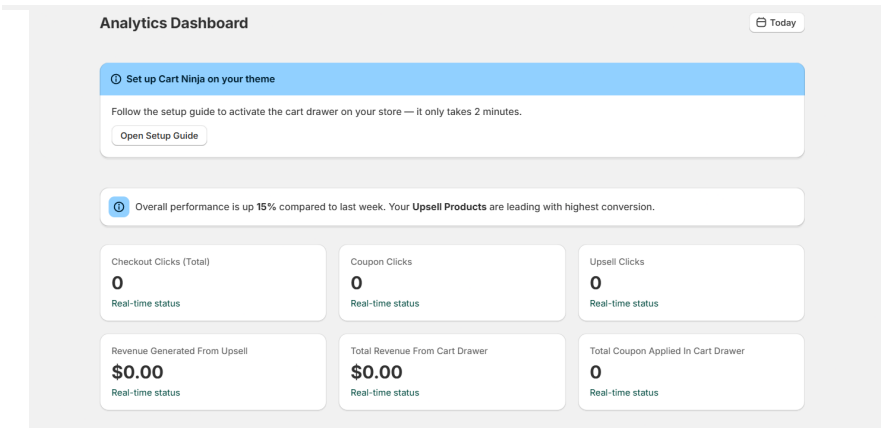
7.7 Subscription Plans (`app.Plan.jsx`)

Three tiers — Free, Pro, Enterprise — presented as side-by-side plan cards with feature comparison. Each card shows the price, a feature matrix with checkmarks/minus icons (Polaris CheckCircleIcon / MinusIcon), and an upgrade/downgrade CTA. Integrates with Shopify Billing API for actual subscription creation.

8. UI SCREENS

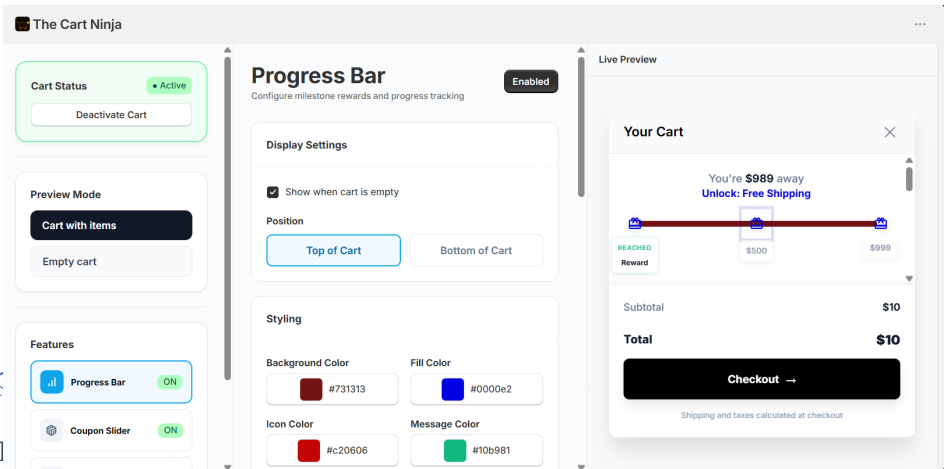
8.1 Main Analytics Dashboard

The home page shows aggregated metrics: total cart revenue, upsell revenue, checkout clicks, coupon clicks, and coupon applications. Charts displayed: Area chart for revenue over time and a composed chart for multi-metric comparison. A date range selector allows filtering by custom periods.



8.2 Cart Drawer Configuration Page

A tabbed interface with three tabs: Progress Bar, Coupons, and Upsell. Features inline live previews of the progress bar via MilestoneProgressBarPreview. Color pickers, range sliders, icon preset selectors, and drag-and-drop tier management.



8.3 Coupon

An Index of coupons (e.g., Free Shipping), start/end dates, usage counts, and status. Supports bulk selection, sorting, and one-click deletion.

Coupons

Export

Create Coupon

Coupon Engine

Manage, create, and view your store coupons in one place.

All (10)Active (6)Scheduled (2)Expired (2)+

Q

Title

Code

Type

Discount

Status

Used

Start date

End date

CODE_DISCOUNT_BLACKFRIDAY

CODE_DISCOUNT_BLACKFRIDAY

Amount off

80% off

Scheduled

0 / ∞

Nov 27, 2026

Nov 28, 2026

CODE_BXGY_DISCOUNT_SUMMERBOGO

CODE_BXGY_DISCOUNT_SUMMERBOGO

Buy X Get Y

BXGY

Scheduled

0 / ∞

Jun 1, 2026

Jul 31, 2026

Test1

6R0P6IXP

Amount off

10% off

Active

1 / ∞

Apr 6, 2026

-

sample

OEKPYKM4

Amount off

10% off

Active

0 / ∞

Apr 6, 2026

-

Buy one, get the second 10 percent off

Buy one, get the second 10 percent off

Buy X Get Y

BXGY

Expired

0 / ∞

Apr 4, 2026

Apr 19, 2026

file:///C:/app_dev/cart-app/app_dev_final_feb/Cart_Ninja_Project_Documentation.html

27/42

8.4 Discount Code Creator

A rich form with sections for discount type selection (percentage/fixed/free shipping), target (products, collections, or entire order), date ranges, usage limits, country restrictions, and minimum order requirements. Uses Polaris DatePicker, Popover, Modal, and ResourceList for product/collection pickers.

The screenshot shows the 'Create discount' form in 'The Cart Ninja' application. The form is titled 'Create discount' and has a 'Save discount' button in the top right. It is divided into several sections:

- Discount type:** Includes buttons for 'Amount off products', 'Amount off order', 'Buy X get Y', and 'Free shipping'.
- Discount code:** A text input field with the example 'e.g. SUMMER2024' and a 'Generate' button.
- Internal title:** A text input field.
- Discount value:** Includes a 'Discount type' dropdown menu (currently set to 'Percentage') and a 'Percentage' input field with a '%' symbol.
- Applies to:** A section for specifying the target of the discount.
- Summary:** A section on the right providing a summary of the discount configuration, including the type, target, code, and start date.
- Discount tip:** A section on the right providing a tip about code-based discounts.

8.5 Upsell Rule Builder

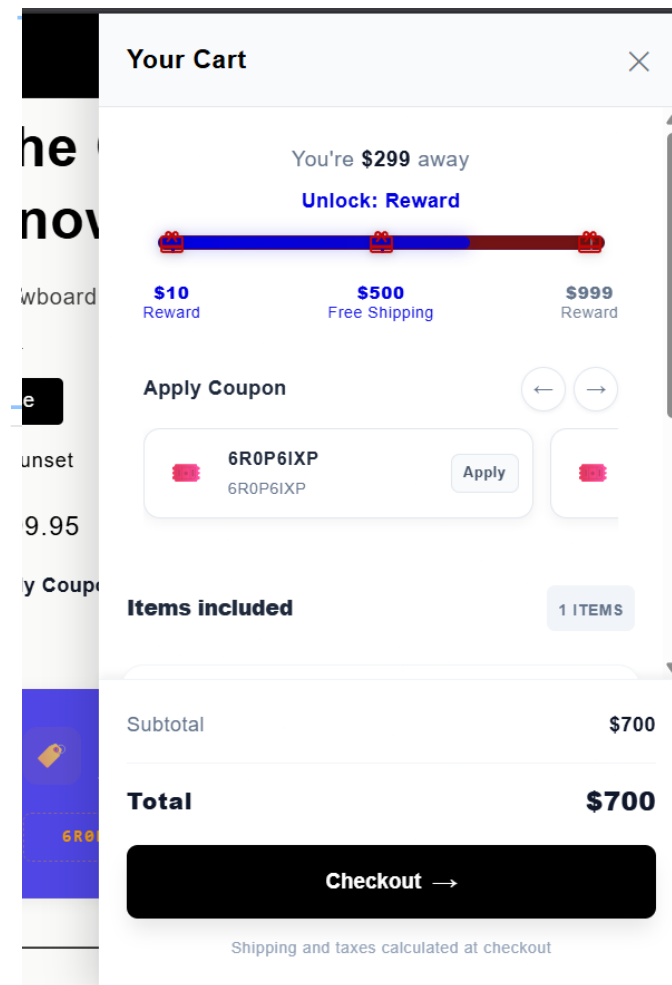
A rule configuration panel where merchants define upsell rules by type (Global, Triggered, Exclusion, Cart Conditions), select trigger and upsell products, set priority, choose layout (slider/vertical), and configure the display title and CTA button text.

8.6 Subscription Plans Page

Three side-by-side plan cards (Free / Pro / Enterprise) with a feature comparison matrix. The most popular plan is highlighted with a `Badge tone="success"` badge. Current plan shows a disabled "Current Plan" button; other plans show "Upgrade" CTA.

8.7 Storefront Cart Drawer (Customer View)

The customer-facing slide-in cart panel showing cart items, the milestone progress bar, coupon cards, and upsell product grid/slider. Fully responsive, brand-configurable colors, and animates smoothly on open/close.



9. WORKFLOW / USER FLOW

9.1 Merchant Onboarding Flow

- 1 **Install App:** Merchant finds Cart Ninja in the Shopify App Store and clicks Install. Shopify redirects to the OAuth flow.
- 2 **OAuth Authorization:** `auth.$.jsx` handles the callback, exchanges the authorization code for an access token, and stores the session via `PrismaSessionStorage`.
- 3 **Dashboard Loads:** Merchant is redirected to `app._index.jsx`. The loader authenticates the session and fetches analytics data from the PHP backend.
- 4 **Configure Cart Drawer:** Merchant navigates to Cart Drawer settings, configures progress bar tiers, coupon display, and upsell rules. Saves configuration via POST to `api.cartdrawer-config.jsx`.
- 5 **Activate Extension:** Merchant enables the Cart Ninja extension block in the Shopify Theme Editor. The extension is now live on the storefront.

9.2 Customer Cart Interaction Flow

- 1 **Add to Cart:** Customer clicks "Add to Cart" on any product page. The Cart Drawer extension intercepts the cart/add event.
- 2 **Fetch Configuration:** The extension calls `/api/cartdrawer-config?shop=...` to load the merchant's latest settings (progress bar, coupons, upsell rules).
- 3 **Render Cart Drawer:** The Cart Drawer slides in showing: cart item list, progress bar with current milestone progress, coupon cards (if enabled), and upsell products (if rules match).
- 4 **Customer Interaction:** Customer may click a coupon (`coupon_click` tracked), click an upsell product add button (`upsell_click` tracked + product added to cart), or click "Checkout" (`checkout_click` tracked).
- 5 **Analytics Event:** Each click sends an event to the PHP analytics backend, incrementing the relevant counter and recording revenue data.

- 6 **Order Placed:** When the customer completes checkout, the `orders/create` webhook fires. Cart Ninja increments the order count and syncs revenue to the external backend via `sync_shop_metrics.php`.

9.3 Coupon Creation Flow

- 1 **Navigate:** Merchant goes to Coupons → Create Discount.
- 2 **Fill Form:** Select discount type, enter code, value, date range, usage limit, eligible products/collections.
- 3 **Submit:** The action function in `app.discounts.create.jsx` calls the appropriate Shopify GraphQL mutation (`discountCodeBasicCreate`, `discountCodeBxgyCreate`, or `discountCodeFreeShippingCreate`).
- 4 **Confirmation:** Success toast is shown; the new discount is immediately visible in the Coupon Manager list and available in the Cart Drawer.

10. API DESIGN

Cart Ninja exposes internal API routes (accessible by both the admin frontend and the storefront extension) using React Router v7 loader/action functions. All admin routes are protected by Shopify OAuth authentication. Storefront routes use shop-domain-based access.

10.1 Cart Drawer Configuration API

Method	Endpoint	Description
GET	/api/cartdrawer-config? shop=:domain	Fetch merged cart drawer configuration for a shop
POST	/api/cartdrawer-config	Save cart drawer configuration (progress bar, coupons, upsell)

GET Response Example:

```
{
  "progressBar": {
    "enabled": true,
    "mode": "amount",
    "barForegroundColor": "#2563eb",
    "tiers": [
      { "id": 1, "minValue": 500, "description": "Free Shipping", "rewardType"
    ]
  },
  "coupons": {
    "enabled": true,
    "selectedStyle": "style-2",
    "position": "top",
    "layout": "grid"
  },
  "upsell": {
    "enabled": true,
    "upsellMode": "manual",
    "activeTemplate": "grid"
  }
}
```

10.2 Analytics API

Method	Endpoint	Description
--------	----------	-------------

GET	/api/analytics? shop=:domain	Proxy to external PHP analytics; returns aggregated metrics
POST	/api/analytics	Record an analytics event (click, revenue)

GET Response Example:

```
{
  "checkout_click": 142,
  "coupon_click": 87,
  "upsell_click": 63,
  "upsell_revenue_generated": 12450.00,
  "cartdrawer_total_revenue": 98700.00,
  "cartdrawer_total_coupon_applied": 54
}
```

10.3 Upsell Rules API

Method	Endpoint	Description
GET	/api/upsell	Fetch all upsell rules + available products for the shop
POST	/api/upsell	Create or update a upsell rule in the Prisma DB
DELETE	/api/upsell?id=:ruleId	Delete a specific upsell rule

10.4 Active Coupons API

Method	Endpoint	Description
GET	/api/coupons-active? shop=:domain	Returns active discount codes to display in the Cart Drawer extension

10.5 Shopify Webhook Handlers

Topic	Route File	Action
orders/create	webhooks.orders.create.jsx	Increment order count in Prisma; sync revenue to PHP backend
app/uninstalled	webhooks.app.uninstalled.jsx	Clean up shop sessions from the database

app/scopes_update	webhooks.app.scopes_update.jsx	Handle permission scope change events
-------------------	--------------------------------	---------------------------------------

11. SHOPIFY APP REVIEW & VERIFICATION PROCESS

11.1 Shopify App Review Preparation

Before submitting Cart Ninja to the Shopify App Store, the application undergoes a structured review preparation workflow to ensure compliance with Shopify Partner requirements, security policies, performance expectations, and merchant usability standards.

- OAuth installation verification
- HMAC validation verification
- Billing flow testing
- Embedded app testing
- Theme extension activation testing
- Mobile responsiveness validation
- Webhook verification
- App uninstall cleanup verification
- Storefront performance validation
- Security testing
- Manual QA testing
- AI-assisted validation

This process was introduced after initial reviewer feedback identified OAuth-related issues during installation verification.

11.2 Shopify Reviewer Environment Preparation

A dedicated Shopify development store is prepared specifically for reviewer testing.

- Preconfigured products
- Active cart drawer extension
- Sample upsell rules
- Active coupon campaigns
- Analytics-enabled environment
- Billing test environment
- Theme extension already deployed

This allows Shopify reviewers to immediately validate the application's functionality without requiring additional setup.

12. OAUTH AUTHENTICATION ARCHITECTURE

12.1 OAuth Installation Flow

Cart Ninja uses Shopify OAuth 2.0 authentication for secure merchant onboarding.

OAuth Flow Steps

- Merchant installs the application from Shopify App Store.
- Shopify redirects merchant to authorization endpoint.
- Merchant approves requested scopes.

- Shopify redirects back with authorization code, shop domain, HMAC signature, and state token.
- Cart Ninja validates the HMAC signature and OAuth state token.
- Access token is exchanged securely.
- Session is persisted using PrismaSessionStorage.
- Merchant is redirected into the embedded application dashboard.

12.2 OAuth Security Validation

OAuth validation includes:

- CSRF prevention using state parameter
- HMAC signature verification
- Session validation
- Secure token persistence
- Expired session handling
- Redirect URI validation

12.3 OAuth Failure Incident (502 Error)

During initial Shopify review, a temporary 502 error occurred during OAuth callback verification.

Root Cause: Instability during session creation and callback processing.

Affected Area: OAuth callback route, session persistence layer, and redirect handling workflow.

Resolution:

- Improved callback validation
- Added fallback error handling
- Added deployment environment validation
- Improved session persistence checks
- Added retry-safe redirect handling

The OAuth installation flow was retested successfully across multiple Shopify development stores, reinstall workflows, uninstall/reinstall cycles, multiple browsers, and mobile environments.

13. HMAC VALIDATION & WEBHOOK SECURITY

13.1 HMAC Verification Purpose

Cart Ninja validates all incoming Shopify requests using HMAC verification to ensure request authenticity and prevent spoofed requests.

HMAC verification protects OAuth callbacks, webhook requests, embedded app communication, and app proxy requests.

13.2 Webhook Verification Workflow

Incoming Shopify webhooks contain the X-Shopify-Hmac-Sha256 header. The backend performs:

- Raw body extraction
- HMAC generation using Shopify secret
- Signature comparison
- Request validation

Invalid webhook requests are rejected silently without exposing system details.

13.3 Webhook Topics

Webhook	Purpose
orders/create	Revenue synchronization
app/uninstalled	Cleanup merchant data
app/scopes_update	Handle permission changes

14. MANUAL QA TESTING PROCESS

14.1 QA Testing Overview

Cart Ninja uses structured manual QA testing before every production deployment and Shopify submission.

Testing validates merchant workflows, storefront rendering, extension compatibility, analytics accuracy, billing functionality, mobile responsiveness, and cross-theme compatibility.

14.2 Merchant Dashboard Testing

Module	Validation
Dashboard Analytics	Metrics render correctly
Cart Drawer Settings	Save/load functionality
Coupon Manager	Discount creation works
Upsell Rules	Rule creation and rendering
Billing	Subscription activation
Navigation	All routes accessible

14.3 Storefront Testing

Feature	Validation
Cart Drawer	Opens correctly

Progress Bar	Calculates milestones
Coupon Slider	Displays active discounts
Upsell Products	Correct rendering
Checkout Tracking	Analytics event fires
Mobile View	Responsive rendering

14.4 Theme Compatibility Testing

Cart Ninja was tested against multiple Shopify themes including Dawn, Debut, Sense, Craft, and custom themes. Validation focused on cart drawer compatibility, AJAX cart integration, DOM stability, and extension rendering consistency.

15. AI-ASSISTED TESTING WORKFLOW

15.1 Purpose

AI-assisted testing was introduced to improve detection of edge cases, security gaps, and logical inconsistencies.

AI systems were used to review OAuth workflows, API security, storefront rendering logic, webhook handling, analytics synchronization, error handling, and database persistence logic.

15.2 AI Validation Areas

Area	Validation Purpose
OAuth	Detect missing validation
APIs	Detect insecure routes
HMAC	Validate request verification
Storefront	Detect rendering inconsistencies
Analytics	Validate tracking events
Database	Detect persistence issues

15.3 AI Edge Case Analysis

AI systems simulated:

- Invalid OAuth callbacks
- Malformed webhook payloads
- Session expiration
- Failed analytics requests
- Missing storefront configuration
- Invalid shop domains
- Duplicate webhook events

This improved overall production stability.

16. DEPLOYMENT VALIDATION PROCESS

16.1 Pre-Deployment Validation

Before deployment:

- Production environment variables validated
- Shopify credentials verified
- Callback URLs verified
- Extension assets built
- Webhooks registered
- Database migrations executed
- Storefront rendering verified

16.2 Post-Deployment Verification

Validation	Expected Result
------------	-----------------

App Installation	Successful
OAuth Flow	Successful
Dashboard Access	Functional
Extension Rendering	Functional
Analytics Tracking	Functional
Billing Flow	Functional

17. SECURITY TESTING PROCEDURES

17.1 Security Validation Areas

Security testing includes:

- Invalid OAuth state rejection
- Invalid HMAC rejection
- Unauthorized API request rejection
- GraphQL injection prevention
- Malformed payload handling
- Environment variable protection

17.2 Input Validation

All user input is validated before persistence or API execution.

Validation includes coupon code format validation, shop domain normalization, product ID validation, JSON payload validation, and GraphQL variable parameterization.

18. FUTURE DEVELOPER ONBOARDING GUIDE

18.1 Purpose

This documentation is designed so future developers can quickly understand the complete Cart Ninja system architecture and operational workflow.

18.2 Mandatory Knowledge Areas

- Shopify OAuth
- Theme App Extensions
- Prisma ORM
- React Router v7
- Shopify GraphQL APIs
- HMAC verification
- Webhook handling
- Embedded app architecture

18.3 Mandatory QA Before Release

Before any production release:

- Manual QA must be completed
- OAuth flow must be verified
- Storefront rendering must be tested
- Extension compatibility must be verified
- Billing flow must be validated
- AI-assisted review recommended

19. FINAL SHOPIFY SUBMISSION CHECKLIST

Requirement	Status
OAuth verified	[]
HMAC validation verified	[]
Extension activation verified	[]
Billing verified	[]
Analytics verified	[]
Webhooks verified	[]
Mobile tested	[]
Cross-theme tested	[]
AI review completed	[]
Manual QA completed	[]
Production deployment verified	[]
Reviewer instructions prepared	[]
Demo store ready	[]
No 500/502 errors	[]

20. SECURITY CONSIDERATIONS

20.1 Authentication & Authorization

Shopify OAuth 2.0

Every admin route starts with `authenticate.admin(request)` which validates the Shopify session token. If the session is invalid or expired, the user is redirected to the OAuth flow. The OAuth state parameter prevents CSRF attacks during the installation flow.

Webhook HMAC Verification

Incoming webhooks are verified using `authenticate.webhook(request)`, which validates the X-Shopify-Hmac-Sha256 header against the app's API secret. Unverified requests throw an empty Response to reject them silently.

Session Storage Security

OAuth access tokens and session data are stored in the Prisma database (server-side), never exposed to the browser. The `PrismaSessionStorage` adapter handles encryption-at-rest when the underlying database supports it.

Environment Variable Protection

All secrets (`SHOPIFY_API_KEY`, `SHOPIFY_API_SECRET`, `SCOPES`) are stored in `.env` which is excluded from version control via `.gitignore`. The API key is never sent to the browser.

20.2 Data Protection

AI Key Sanitization

The `stripLegacyAiKeysFromObject()` function in `api.cartdrawer-config.jsx` actively removes any AI API keys (`aiApiKey`, `openaiKey`, etc.) from configuration objects before they are persisted or returned, preventing accidental key leakage.

Input Validation

Form inputs are validated server-side in action functions before being passed to GraphQL mutations. The `app/validators/` directory contains reusable validation helpers. User-supplied coupon codes and product IDs are validated against expected formats.

GraphQL Injection Prevention

All GraphQL queries use parameterized variables (`{ variables: { id } }`) rather than string interpolation, preventing GraphQL injection attacks.

Shop Domain Normalization

The `normalizeShopDomain()` function lowercases and trims all shop domain inputs before database queries, preventing domain spoofing via case manipulation.

20.3 CORS & CSP

The Shopify App Bridge enforces Content Security Policy headers for the embedded admin app. The Cart Drawer extension runs in the store's own origin context, inheriting the store's security policies. API routes that serve the storefront extension set permissive CORS headers only for storefront-facing endpoints (e.g., `/api/cartdrawer-config`), while admin API routes rely solely on Shopify session token validation.

21. PERFORMANCE OPTIMIZATION

21.1 Frontend Optimizations

Technique	Implementation
React Memoization	<code>useMemo</code> and <code>useCallback</code> are used extensively in cart drawer config forms and coupon list components to prevent unnecessary re-renders on large option sets.
Lazy State Initialization	Complex initial state objects (coupon styles, upsell rule defaults) are computed once via factory functions rather than re-created on each render.
Recharts Responsiveness	All charts are wrapped in <code>ResponsiveContainer</code> , allowing browser-native resize handling without JavaScript resize observers.
Polaris Component Reuse	Polaris components are imported individually (tree-shakeable imports) to keep bundle size minimal. No full-library imports.
Date Picker Popover	Date pickers use Polaris <code>Popover</code> (portal rendering) to avoid layout shifts and render only when opened.

21.2 Backend & API Optimizations

Technique	Implementation
Dual-Storage Fallback	Cart drawer config is read from the local JSON file first if the external PHP API is slow or unavailable. This eliminates user-perceived latency from network calls.
Parallel Promise Resolution	In the analytics route, multiple upstream API URLs are tried in parallel using <code>Promise.any()</code> , returning as soon as the fastest responds.
Prisma Upsert	The <code>orders/create</code> webhook uses <code>prisma.orderCount.upsert()</code> to atomically increment counters, avoiding race conditions under high order volume.
Indexed Queries	The <code>UpsellRule</code> model defines <code>@@index([shop])</code> and <code>@@index([ruleType])</code> , ensuring fast rule lookups when filtering by shop and rule type.

Response Caching	GraphQL product queries (product picker) return up to 50 products per request with cursor-based pagination, avoiding over-fetching.
---------------------	---

21.3 Storefront Extension Optimizations

Technique	Implementation
Minimal JavaScript	Extension assets are plain Vanilla JS (no framework), keeping the bundle under 30KB gzipped to avoid impacting storefront Core Web Vitals.
Lazy Fetch	The cart drawer configuration is fetched only when the drawer opens for the first time, not on page load. Subsequent opens reuse the cached config.
CSS-Only Animations	The drawer open/close animation uses CSS transforms and transitions, avoiding JavaScript animation frame loops.
Event Delegation	Click tracking uses event delegation on the drawer container rather than individual event listeners per element, reducing memory overhead.

21.4 Build-Time Optimizations

Vite 6 is used as the build tool with React Router's build pipeline. Production builds use code splitting at the route level, TypeScript path aliases for clean imports, and tree-shaking for unused code elimination. The vite-tsconfig-paths plugin ensures alias resolution works correctly during build.

22. CHALLENGES FACED

1. Dual-Storage Consistency

Keeping the local JSON fallback file and the external PHP backend in sync proved complex. When the PHP backend was temporarily unavailable, local writes could diverge. Solved by implementing a merge strategy: PHP backend is the source of truth; local file acts as a read-through cache that is invalidated on every successful PHP write.

2. Shopify OAuth Session Expiry

Shopify's expiring offline access token feature introduced session expiry handling complexity. The `future.expiringOfflineAccessTokens: true` flag required adding refresh token storage fields to the Session model and implementing token refresh logic in the authentication middleware.

3. Cart Drawer Extension Compatibility

Different Shopify themes handle the cart in different ways (drawer, page, popup). Ensuring the Theme App Extension block worked correctly across Dawn, Debut, and custom themes required testing with multiple layouts and implementing theme-agnostic DOM selectors.

4. GraphQL Discount Mutation Complexity

Shopify's GraphQL API uses separate mutations per discount type (Basic, BxGy, FreeShipping). There is no generic "create discount" mutation, requiring the form to track the selected type and route to the correct mutation. Similarly, deletions need the correct typed mutation.

5. Analytics Data Normalization

The PHP analytics backend returns fields in different naming conventions across endpoints (snake_case vs camelCase). A normalization layer (`extractCounts()`) was built to handle all known field name variants, ensuring the dashboard always displays accurate data.

6. React Router v7 Migration from Remix

The project was initialized with the React Router v7 package (which subsumes Remix). Some Shopify documentation and community examples still reference Remix APIs. Mapping Remix

patterns (loaders, actions, meta) to their React Router v7 equivalents required careful attention to API differences.

23. FUTURE ENHANCEMENTS

- **A/B Testing Engine:** Allow merchants to split-test different cart drawer configurations (e.g., two upsell layouts) and measure conversion rate differences.
- **AI Product Recommendations v2:** Integrate with a dedicated ML recommendation engine trained on the merchant's actual purchase history data.
- **Multi-language Support (i18n):** Extend the extension's locale files to support dynamic language switching based on the shopper's locale.
- **Cart Abandonment Recovery:** Detect when a shopper closes the cart drawer without checking out and trigger a re-engagement notification or discount popup.
- **Advanced Analytics:** Add cohort analysis, funnel visualization (add-to-cart → drawer open → coupon applied → checkout), and per-product upsell performance breakdowns.
- **Headless Commerce Support:** Provide a JavaScript SDK and REST API so Cart Ninja can be used with headless Shopify storefronts (Hydrogen, Next.js).
- **PostgreSQL Production Database:** Migrate from SQLite to a managed PostgreSQL instance for production deployments with connection pooling via PgBouncer.
- **Staff Dashboard:** Build a separate read-only analytics view for customer support teams with ticket-level cart session replay.
- **Mobile App for Merchants:** A React Native or PWA merchant app to view real-time analytics and push-configure the cart drawer on mobile.
- **Shopify Flow Integration:** Expose Cart Ninja events as Shopify Flow triggers to allow merchants to automate workflows (e.g., "when upsell revenue exceeds \$500/day, send Slack alert").

24. CONCLUSION

Cart Ninja represents a comprehensive, production-ready Shopify application that addresses a real and measurable problem in e-commerce: cart-stage revenue leakage. By combining an intelligent Cart Drawer, a Coupon Engine, a rule-based Upsell Engine, and a real-time Analytics Dashboard into a single, unified platform, Cart Ninja gives Shopify merchants the tools they need to maximize average order value and conversion rate without sacrificing storefront performance.

The technical architecture reflects modern best practices: server-side rendering via React Router v7 for fast, SEO-friendly admin pages; Prisma ORM for type-safe, migration-driven database management; Shopify's official design system (Polaris) for a consistent and accessible merchant UI; and a lightweight Vanilla JS Theme App Extension for a near-zero-impact storefront footprint.

The development of Cart Ninja provided deep practical experience in Shopify app development, OAuth integration, GraphQL API design, webhook handling, real-time analytics, and the challenges of maintaining consistency across distributed data stores. The dual-storage pattern, AI key sanitization, and analytics normalization layer are particularly notable engineering decisions that demonstrate maturity in handling real-world edge cases.

With a solid foundation in place, Cart Ninja is well-positioned for the future enhancements outlined in this document — particularly headless commerce support, A/B testing, and advanced cohort analytics. The modular architecture ensures that new features can be added as independent modules without disrupting existing functionality.

Cart Ninja is a complete, functional, and deployable Shopify application. This documentation serves as both an academic record and a practical technical reference for ongoing development, onboarding of new contributors, and merchant support.

25. REFERENCES

1. Shopify. (2025). *Shopify App Development Documentation*. Shopify Developer Platform. <https://shopify.dev/docs/apps>
2. Shopify. (2025). *Shopify Admin GraphQL API Reference (2025-10)*. <https://shopify.dev/docs/api/admin-graphql>
3. Shopify. (2025). *Theme App Extensions*. <https://shopify.dev/docs/apps/online-store/theme-app-extensions>
4. Shopify. (2025). *Polaris Design System*. <https://polaris.shopify.com>

5. Shopify. (2025). *@shopify/shopify-app-react-router SDK*.
<https://github.com/Shopify/shopify-app-js>
6. Prisma. (2025). *Prisma ORM Documentation*. <https://www.prisma.io/docs>
7. React Router. (2025). *React Router v7 Documentation*. <https://reactrouter.com>
8. Meta / React. (2025). *React 18 Documentation*. <https://react.dev>
9. Recharts. (2025). *Recharts Documentation*. <https://recharts.org>
10. Vite. (2025). *Vite Build Tool Documentation*. <https://vitejs.dev>
11. OWASP. (2024). *OWASP Top 10 Web Application Security Risks*. <https://owasp.org/www-project-top-ten>
12. Shopify. (2025). *App Bridge Documentation*. <https://shopify.dev/docs/api/app-bridge>

— End of Document —

Cart Ninja — Shopify Cart Optimization App | Project Documentation v1.0 | April 2026

Prepared by : Abilashmi.K | Company Name : Digifyce |